

Raport z Projektu: Przewidywanie awarii maszyn (Predictive Maintenance)

Autorzy: Oliwier Trąbczyński, Bartosz Wankiewicz

1. Cel Biznesowy i Problem Badawczy

Głównym celem projektu było opracowanie modelu zdolnego do predykcji usterek maszyn przemysłowych (frezarek CNC) na podstawie danych telemetrycznych z czujników.

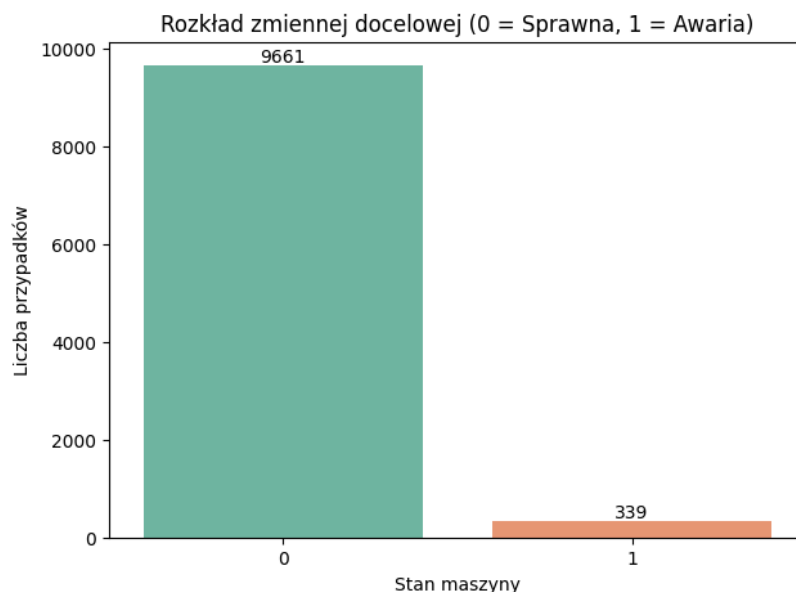
Rozwiązuje to fundamentalny problem utrzymania ruchu: przejście z kosztownej konserwacji reaktywnej (naprawa po zniszczeniu maszyny) na **Predictive Maintenance** (zapobieganie awariom z wyprzedzeniem). Analizie poddano syntetyczny zbiór AI4I 2020, symulujący realne parametry, takie jak temperatura procesu czy prędkość obrotowa.

2. Wyzwanie Techniczne: Paradoks Dokładności

Kluczową barierą w projekcie był drastyczny brak balansu klas.

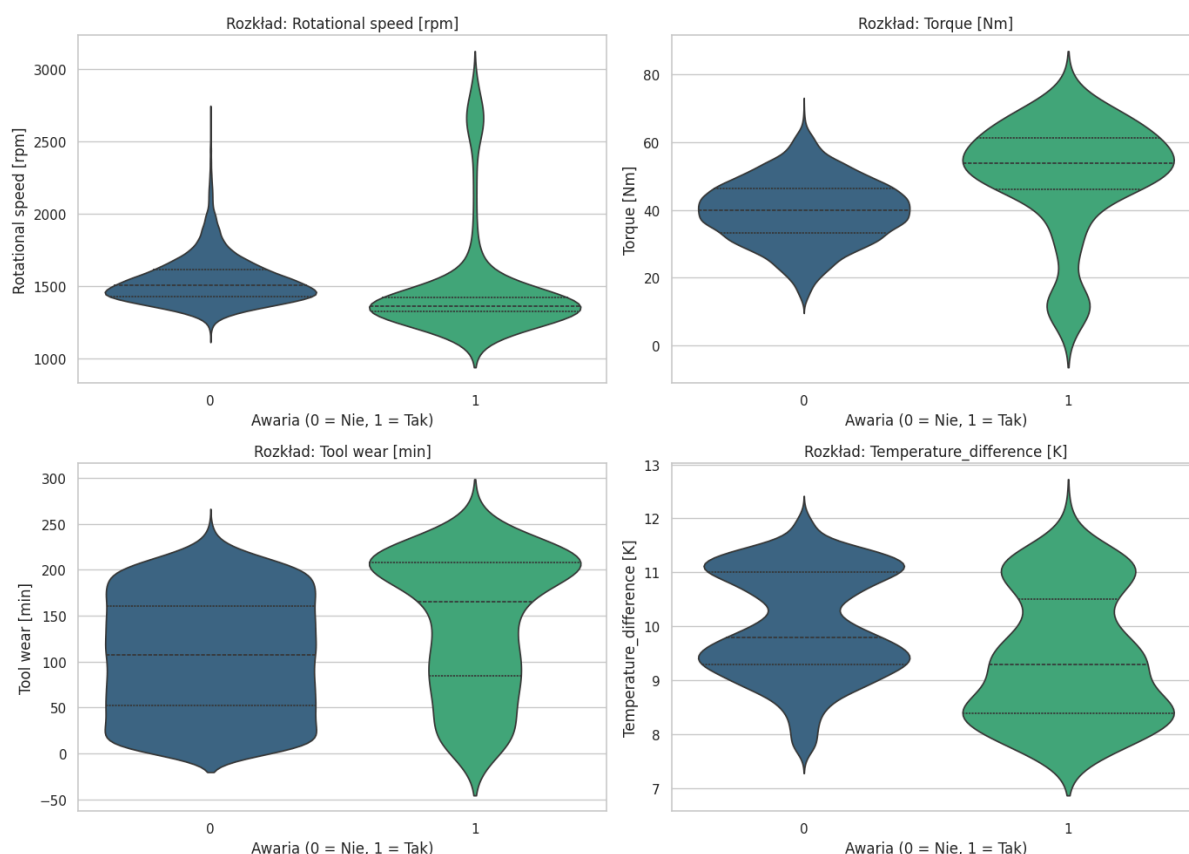
- Maszyny sprawne stanowią 96.6% zbioru.
- Awarie stanowią zaledwie 3.4% zbioru.

W takich warunkach klasyczna metryka *Accuracy* (Dokładność) jest bezużyteczna. Algorytm, który całkowicie zignoruje odczyty i zawsze orzeknie "maszyna sprawna", osiągnąłby 96.6% dokładności, przegapiając przy tym wszystkie rzeczywiste usterki. Z tego powodu głównym celem optymalizacyjnym stała się **Czułość (Recall)** – zdolność modelu do wyłapywania jak największej liczby rzadkich anomalii.



3. Analiza Eksploracyjna: Fizyczny obraz awarii

Przed przystąpieniem do budowy modeli predykcyjnych, kluczowe było zrozumienie natury samych awarii. W tym celu przeprowadzono analizę rozkładów najważniejszych zmiennych ciągłych w podziale na stan maszyny (sprawna vs awaria).



Wnioski z analizy wizualnej: Rozkłady gęstości jednoznacznie potwierdzają, że awarie nie mają charakteru losowego – są silnie skorelowane ze skrajnymi odczytami z czujników telemetrycznych:

- **Prędkość obrotowa (Rotational speed):** Rozkład dla maszyn uszkodzonych jest spłaszczony i rozciągnięty. Awarie korelują ze skrajnie niskimi lub bardzo wysokimi prędkościami, podczas gdy maszyny sprawne pracują w stabilnym, wąskim przedziale.
- **Moment obrotowy (Torque):** Awarie występują przy znacznie wyższej wariancji (rozrzucie) siły skręcającej, co sugeruje trudności w obróbce twardego materiału.
- **Zużycie narzędzia (Tool wear):** To najsilniejszy wizualny wskaźnik. Mediana czasu pracy dla maszyn, które uległy awarii, jest drastycznie przesunięta w górę. Zbyt długa praca jednym narzędziem (np. tępym frezem) krytycznie zwiększa ryzyko usterki.

Dzięki tej analizie zysaliśmy pewność, że w danych ukryty jest silny, nieliniowy wzorzec, z którym doskonale powinny poradzić sobie modele oparte na drzewach decyzyjnych.

4. Wybór i mechanika Modelu: Las Losowy vs XGBOOST

W fazie implementacji zderzono ze sobą dwa potężne algorytmy oparte na zespołach drzew decyzyjnych: Random Forest (wykorzystujący mechanizm baggingu) oraz XGBoost (wykorzystujący boosting). W obu modelach zastosowano techniki wagowania klas (class_weight oraz scale_pos_weight), aby zmusić algorytmy do kładzenia nacisku na rzadko występujące awarie.

```
=====
WYNIKI DLA MODELU: XGBOOST
=====
              precision    recall  f1-score   support

      0       0.99         0.99         0.99     1932
      1       0.82         0.82         0.82         68

   accuracy                0.99     2000
  macro avg       0.91         0.91         0.91     2000
 weighted avg       0.99         0.99         0.99     2000

=====
WYNIKI DLA MODELU: RANDOM FOREST (Las Losowy)
=====
              precision    recall  f1-score   support

      0       0.99         1.00         0.99     1932
      1       0.94         0.69         0.80         68

   accuracy                0.99     2000
  macro avg       0.96         0.84         0.90     2000
 weighted avg       0.99         0.99         0.99     2000
```

Wnioski z porównania i mechanika algorytmu: Mimo zastosowania zbalansowanych wag, Random Forest okazał się niewystarczający w warunkach silnego niezbalansowania danych – osiągnął Czulość (Recall) dla klasy awarii na poziomie jedynie 0.69 (przeoczył aż 31% usterek). Z kolei XGBoost (Model Bazowy) poradził sobie znacznie lepiej, osiągając Czulość rzędu 0.82.

Dlaczego XGBoost wygrał to starcie? Zamiast budować jeden skomplikowany model, tworzy on zespół (sztafetę) wielu prostych drzew decyzyjnych. Każde kolejne drzewo w sekwencji uczy się na błędach swojego poprzednika, skupiając się na tych maszynach, które wcześniej zostały źle sklasyfikowane. Sekwencyjny sposób uczenia algorytmu XGBoost okazał się idealny do wyłapywania trudnych, nieliniowych anomalii z czujników. Dzięki wbudowanemu parametrowi obsługi wag klas, model został skutecznie zmuszony do priorytetyzowania

mniejszościowych przypadków awarii. Z tego względu to on został wytypowany jako model docelowy.

5. Optimalizacja Hiperparametrów (GridSearchCV)

Mając wybranego faworyta (XGBoost), podjęto próbę wyśrubowania jego zdolności do wykrywania awarii, wykorzystując metodę przeszukiwania siatki parametrów (GridSearchCV).

Kluczowe założenia eksperymentu:

1. Zastosowano walidację krzyżową ze stratyfikacją (**StratifiedKfold**), co zagwarantowało, że model podczas uczenia w każdym z podzbiorów testowych widział dokładnie taki sam odsetek awarii (3.4%).
2. Siatka była optymalizowana bezpośrednio pod kątem maksymalizacji metryki **Recall**, a nie domyślnej dokładności.

Algorytm przetestował dziesiątki kombinacji i wyłonił architekturę opartą na płytkich drzewach (**max_depth=3**) oraz niskim tempie uczenia (**learning_rate=0.01**). Ta modyfikacja pozwoliła na drastyczny skok skuteczności: model zoptymalizowany podniósł Czułość z 82% do imponujących **93%**.

```
Najlepsze znalezione parametry: {'learning_rate': 0.01, 'max_depth': 3, 'n_estimators': 100}
```

```
=====
WYNIKI DLA ZOPTYMALIZOWANEGO MODELU XGBOOST
=====
```

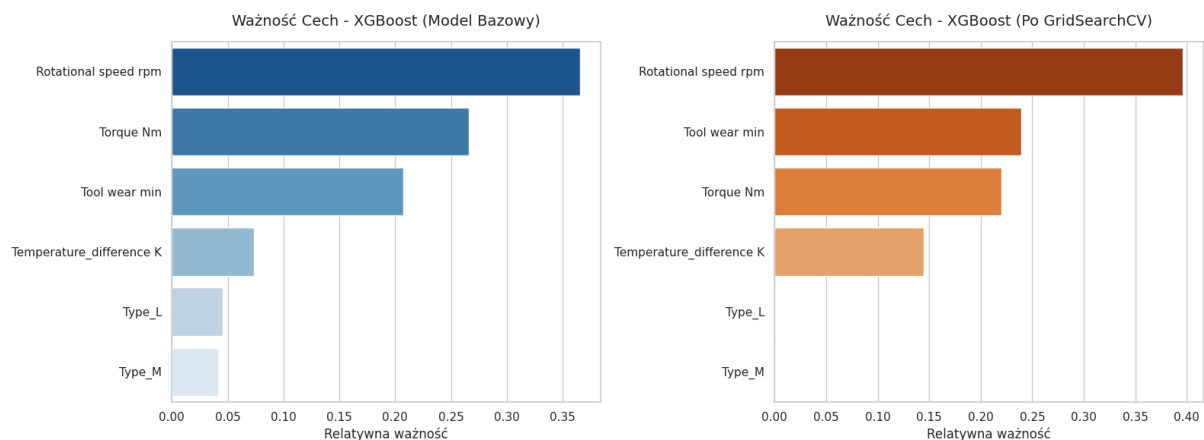
	precision	recall	f1-score	support
0	1.00	0.91	0.95	1932
1	0.26	0.93	0.40	68
accuracy			0.91	2000
macro avg	0.63	0.92	0.68	2000
weighted avg	0.97	0.91	0.93	2000

Wybór wariantów do ostatecznej ewaluacji: Od tego momentu, do finalnej analizy interpretacyjnej i biznesowej wytypowano dwa konkretne "wcielenia" zwycięskiego algorytmu, aby sprawdzić, jak optymalizacja wpłynęła na jego decyzje:

- **XGBoost (Model Bazowy):** Korzystający z domyślnych ustawień, stanowiący stabilny punkt odniesienia.
- **XGBoost (Po GridSearchCV):** Ekstremalnie "czuły", wyśrubowany algorytm ratujący najwięcej maszyn.

6. Interpretowalność: Co faktycznie powoduje awarie?

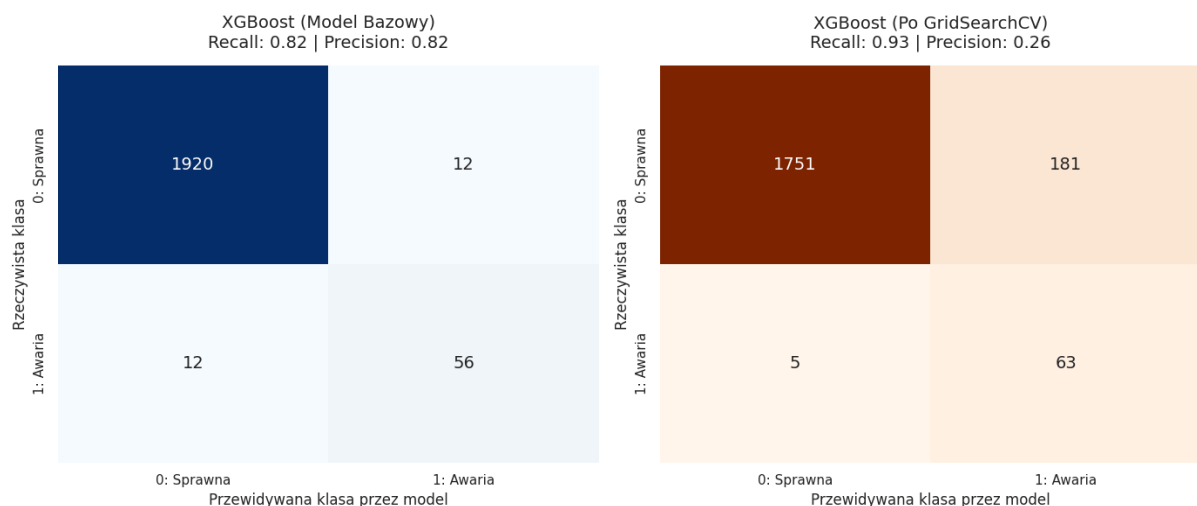
Wiedza o tym, *dlaczego* model podejmuje decyzje, jest kluczowa dla inżynierów. Przeprowadzono analizę ważności cech (Feature Importance) przed i po optymalizacji hiperparametrów za pomocą metody GridSearchCV.



Wnioski z wykresu: Model zoptymalizowany (wykorzystujący płytkie drzewa o głębokości równej 3) musiał dokonać ostrej selekcji informacji. Całkowicie zignorował on kategorię jakości producenta, redukując ich wagę do zera. Algorytm udowodnił, że twarda fizyka z czujników (prędkość obrotowa, zużycie narzędzia tnącego, moment obrotowy i różnica temperatur) jest w zupełności wystarczająca do przewidzenia zbliżającej się awarii.

7. Kompromis Biznesowy: Precyzja vs Czułość

Ostatnim etapem projektu była ewaluacja biznesowa. W uczeniu maszynowym zwiększenie Czułości (wykrywanie większej liczby awarii) zawsze pociąga za sobą spadek Precyzji (generowanie fałszywych alarmów).



Macierze pomyłek obrazują wynik działania dwóch strategii na zbiorze testowym:

- **Strategia 1 (Model Bazowy):** Wyłapuje 82% awarii, generując przy tym minimalną liczbę fałszywych alarmów. To stabilny "złoty środek".

- **Strategia 2 (Model Zoptymalizowany):** Działa jak czuły alarm dymny. Wyłapuje aż 93% awarii, ale kosztem drastycznego wzrostu fałszywych alarmów (precyzja spada do 26%).

8. Symulacja Finansowa i Wniosek Końcowy

Czy warto było tak drastycznie obniżać precyzję modelu zoptymalizowanego? Odpowiedź daje symulacja kosztów operacyjnych.

Zakładając koszt krytycznej usterki (np. spalenie wrzeciona) na poziomie 15 000 PLN oraz koszt weryfikacji fałszywego alarmu na 200 PLN, otrzymujemy następujące straty dla rozpatrywanej partii maszyn:

- **Strata przy modelu bazowym:** 182 400 PLN (przez większą liczbę przeoczonych awarii).
- **Strata przy modelu zoptymalizowanym:** 110 800 PLN (ratujemy więcej maszyn kosztem tanich wizyt technika).

Rekomendacja końcowa: Hipoteza badawcza została potwierdzona. Modele nie muszą być perfekcyjne we wszystkich metrykach. Celowe podbicie Czułości na rzadkich danych telemetrycznych pozwala na zaoszczędzenie ponad 70 000 PLN, udowadniając, że dobrze skalibrowany model to bezpośrednie narzędzie optymalizacji finansowej zakładu.